



Chapter 08: Inheritance

Object Oriented Programming Language (JAVA)

Md. Eamin Rahman



Inheritance Basics

- **Inheritance** allows the creation of **hierarchical classifications**.
- In the terminology of Java, a **class** that is **inherited** is called a **superclass**. The class that does the **inheriting** is called a **subclass**.
- Therefore, a **subclass** is a specialized version of a **superclass**. It **inherits** all of the **members** defined by the **superclass** and adds its own, **unique** elements.

Inheritance Basics

- To **inherit** a class, you simply incorporate the definition of one class into another by using the **extends** keyword.
- The general form of a class declaration that inherits a superclass is shown here:

```
1 class subclass-name extends superclass-name {  
2     // body of class  
3 }
```

Inheritance Basics

- Being a **superclass** for a **subclass** does not mean that the **superclass** cannot be used by itself. Further, a **subclass** can be a **superclass** for another **subclass**.
- You can only specify one **superclass** for any **subclass** that you create. Java does not support the inheritance of multiple **superclasses** into a single **subclass**.
- However, no **class** can be a **superclass** of itself.

A Simple Inheritance Example

```
1 // Create a superclass.
2 class A {
3     int i, j;
4     void showij() {
5         IO.println("i and j: " + i + " " + j);
6     }
7 }
```

```
1 // Create a subclass by extending class A.
2 class B extends A {
3     int k;
4     void showk() {
5         IO.println("k: " + k);
6     }
7     void sum() {
8         IO.println("i+j+k: " + (i+j+k));
9     }
10 }
```

A Simple Inheritance Example

```
1 // Demonstrate inheritance.
2 class SimpleInheritance {
3     public static void main(String[] args) {
4         A superOb = new A();
5         B subOb = new B();
6         // The superclass may be used by itself.
7         superOb.i = 10;          superOb.j = 20;
8         IO.println("Contents of superOb: ");
9         superOb.showij();
10        // The subclass has access to all public members of its superclass.
11        subOb.i = 7;              subOb.j = 8;          subOb.k = 9;
12        IO.println("Contents of subOb: ");
13        subOb.showij();
14        subOb.showk();
15        IO.println("Sum of i, j and k in subOb:");
16        subOb.sum();
17    }
18 }
```

A Simple Inheritance Example

```
1 // Demonstrate inheritance.
2 class SimpleInheritance {
3     public static void main(String[] args) {
4         A superOb = new A();
5         B subOb = new B();
6         // The superclass may be used by itself.
7         superOb.i = 10;          superOb.j = 20;
8         IO.println("Contents of superOb: ");
9         superOb.showij();
10        // The subclass has access to all public members of its superclass.
11        subOb.i = 7;              subOb.j = 8;          subOb.k = 9;
12        IO.println("Contents of subOb: ");
13        subOb.showij();
14        subOb.showk();
15        IO.println("Sum of i, j and k in subOb:");
16        subOb.sum();
17    }
18 }
```

Console Output

```
Contents of superOb:
i and j: 10 20
Contents of subOb:
i and j: 7 8
k: 9
Sum of i, j and k in subOb:
i+j+k: 24
```

Member Access and Inheritance

- Although a **subclass** includes all of the members of its **superclass**
- It cannot access those members of the **superclass** that have been declared as **private**.

Member Access and Inheritance

```
1 class A {  
2     int i; // default access  
3     private int j; // private  
4     void setij(int x, int y) {  
5         i = x;  
6         j = y;  
7     }  
8 }
```

```
1 // A's j is not accessible here.  
2 class B extends A {  
3     int total;  
4     void sum() {  
5         // j is not accessible here  
6         total = i + j; // ERROR,  
7     }  
8 }
```

```
1 // Demonstrate access control.  
2 class Access {  
3     public static void main(String[] args) {  
4         B subOb = new B();  
5         subOb.setij(10, 12);  
6         subOb.sum();  
7         IO.println("Total is " + subOb.total);  
8     }  
9 }
```

A More Complex Inheritance Example

```
1 // This program uses inheritance to extend Box.
2 class Box {
3     double width,height,depth;
4     Box(Box ob) { // pass object to constructor
5         width = ob.width; height = ob.height; depth = ob.depth;
6     }
7     Box(double w, double h, double d) { width = w; height = h; depth = d; }
8     Box() { width = -1; height = -1; depth = -1; }
9     Box(double len) { width = height = depth = len; }
10    double volume() { return width * height * depth; }
11 }
```

A More Complex Inheritance Example

```
1 class BoxWeight extends Box {  
2     double weight; // weight of box  
3     BoxWeight(double w, double h, double d, double m) {  
4         width = w;  
5         height = h;  
6         depth = d;  
7         weight = m;  
8     }  
9 }
```

A More Complex Inheritance Example

```
1  class DemoBoxWeight {
2      public static void main(String[] args) {
3          BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
4          BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
5          double vol;
6          vol = mybox1.volume();
7          IO.println("Volume of mybox1 is " + vol);
8          IO.println("Weight of mybox1 is " + mybox1.weight);
9          IO.println();
10         vol = mybox2.volume();
11         IO.println("Volume of mybox2 is " + vol);
12         IO.println("Weight of mybox2 is " + mybox2.weight);
13     }
14 }
```

A More Complex Inheritance Example

```
1 // Here, Box is extended to include color.
2 class ColorBox extends Box {
3     int color; // color of box
4     ColorBox(double w, double h, double d, int c) {
5         width = w;
6         height = h;
7         depth = d;
8         color = c;
9     }
10 }
```

A Superclass Variable Can Reference a Subclass Object

- A reference variable of a **superclass** can be assigned a reference to any **subclass** derived from that **superclass**.
- You will find this aspect of **inheritance** quite useful in a variety of situations.
- **Superclass** references can only access those members defined by the **superclass**.

A Superclass Variable Can Reference a Subclass Object

```
1  class RefDemo {
2      public static void main(String[] args) {
3          BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
4          Box plainbox = new Box();
5          IO.println("Volume of weightbox is " + weightbox.volume());
6          IO.println("Weight of weightbox is " + weightbox.weight);
7          IO.println();
8          // assign BoxWeight reference to Box reference
9          plainbox = weightbox;
10         // The following is OK, volume() defined in Box
11         IO.println("Volume of plainbox is " + plainbox.volume());
12         IO.println("Volume of plainbox is " + vol);
13         /* The following statement is invalid because plainbox
14         does not define a weight member. */
15         // IO.println("Weight of plainbox is " + plainbox.weight);
16     }
17 }
```

Using **super**

- Whenever a **subclass** needs to refer to its immediate **superclass**, it can do so by use of the keyword **super**.
- **super** has **two** general forms.
- The first calls the **superclass**' constructor.
- The second is used to access a member of the **superclass** that has been hidden by a member of a **subclass**.

Using `super` to Call Superclass Constructors

- A subclass can call a constructor defined by its superclass by use of the following form of `super`:

```
super(arg-list);
```

- Here, `arg-list` specifies any arguments needed by the constructor in the superclass.
- `super( )` must always be the **first statement** executed inside a subclass' constructor.

An Example of Using **super** to Call a Superclass Constructor

```
1 // BoxWeight now uses super to initialize its Box attributes.
2 class BoxWeight extends Box {
3     double weight; // weight of box
4     // initialize width, height, and depth using super()
5     BoxWeight(double w, double h, double d, double m) {
6         super(w, h, d); // call superclass constructor
7         weight = m;
8     }
9 }
```

An Example of Using **super** to Call a Superclass Constructor

```
1 // BoxWeight now uses super to initialize its Box attributes.
2 class BoxWeight extends Box {
3     double weight; // weight of box
4     // initialize width, height, and depth using super()
5     BoxWeight(double w, double h, double d, double m) {
6         super(w, h, d); // call superclass constructor
7         weight = m;
8     }
9 }
```

A Second Use for `super`

- The **second** form of `super` acts somewhat like this, except that it always refers to the **superclass** of the **subclass** in which it is used.
- This usage has the following general form: `super.member`
- Here, **member** can be either a method or an instance variable.
- This second form of `super` is most applicable to situations in which member names of a **subclass** hide members by the same name in the **superclass**.

An Example of Using **super** to Access a Superclass Member

```
1 class A {
2     int i;
3 }
4 class B extends A {
5     int i; // this i hides the i in A
6     B(int a, int b) {
7         super.i = a; // i in A
8         i = b; // i in B
9     }
10    void show() {
11        IO.println("i in superclass: " + super.i);
12        IO.println("i in subclass: " + i);
13    }
14 }
```

An Example of Using **super** to Access a Superclass Member

```
1 class A {
2     int i;
3 }
4 class B extends A {
5     int i; // this i hides the i in A
6     B(int a, int b) {
7         super.i = a; // i in A
8         i = b; // i in B
9     }
10    void show() {
11        IO.println("i in superclass: " + super.i);
12        IO.println("i in subclass: " + i);
13    }
14 }
```

An Example of Using `super` to Access a Superclass Member

```
1 class A {
2     int i;
3 }
4 class B extends A {
5     int i; // this i hides the i in A
6     B(int a, int b) {
7         super.i = a; // i in A
8         i = b; // i in B
9     }
10    void show() {
11        IO.println("i in superclass: " + super.i);
12        IO.println("i in subclass: " + i);
13    }
14 }
```

An Example of Using **super** to Access a Superclass Member

```
1 class UseSuper {  
2     public static void main(String[] args) {  
3         B subOb = new B(1, 2);  
4         subOb.show();  
5     }  
6 }
```

An Example of Using `super` to Access a Superclass Member

```
1 class UseSuper {  
2     public static void main(String[] args) {  
3         B subOb = new B(1, 2);  
4         subOb.show();  
5     }  
6 }
```

Console Output

```
i in superclass: 1  
i in subclass: 2
```

Creating a Multilevel Hierarchy

```
1 class Box {
2     private double width;
3     private double height;
4     private double depth;
5     Box(Box ob) { // pass object to constructor
6         width = ob.width; height = ob.height; depth = ob.depth;
7     }
8     Box(double w, double h, double d) { width = w; height = h; depth = d; }
9     Box() { width = -1; height = -1; depth = -1; }
10    Box(double len) { width = height = depth = len; }
11    double volume() { return width * height * depth; }
12 }
```

Creating a Multilevel Hierarchy

```
1 class BoxWeight extends Box {  
2     double weight; // weight of box  
3     BoxWeight(BoxWeight ob) { super(ob); weight = ob.weight; }  
4     BoxWeight(double w, double h, double d, double m) {  
5         super(w, h, d); weight = m; }  
6     BoxWeight() { super(); weight = -1; }  
7     BoxWeight(double len, double m) { super(len); weight = m; }  
8 }
```

Creating a Multilevel Hierarchy

```
1    // Add shipping costs.
2    class Shipment extends BoxWeight {
3        double cost;
4        Shipment(Shipment ob) { super(ob); cost = ob.cost; }
5        Shipment(double w, double h, double d, double m, double c) {
6            super(w, h, d, m); cost = c; }
7        Shipment() { super(); cost = -1; }
8        Shipment(double len, double m, double c) {
9            super(len, m); cost = c; }
10 }
```

Creating a Multilevel Hierarchy

```
1 class DemoShipment {
2     public static void main(String[] args) {
3         Shipment shipment1 = new Shipment(10, 20, 15, 10, 3.41);
4         Shipment shipment2 = new Shipment(2, 3, 4, 0.76, 1.28);
5         double vol;
6         vol = shipment1.volume();
7         IO.println("Volume of shipment1 is " + vol);
8         IO.println("Weight of shipment1 is " + shipment1.weight);
9         IO.println("Shipping cost: $" + shipment1.cost);
10        IO.println();
11        vol = shipment2.volume();
12        IO.println("Volume of shipment2 is " + vol);
13        IO.println("Weight of shipment2 is " + shipment2.weight);
14        IO.println("Shipping cost: $" + shipment2.cost);
15    }
16 }
```

When Constructors Are Executed

- The answer is that in a class **hierarchy**, constructors complete their **execution** in order of derivation, from **superclass** to **subclass**.
- Further, since `super( )` must be the first statement executed in a **subclass**' constructor, this order is the same whether or not `super( )` is used.
- If `super( )` is not used, then the **default** or parameterless constructor of each **superclass** will be executed.

An Example of Constructor Execution Order

```
1 class A {  
2     A() { IO.println("Inside A's constructor."); }  
3 }
```

```
1 class B extends A {  
2     B() { IO.println("Inside B's constructor."); }  
3 }
```

```
1 class C extends B {  
2     C() { IO.println("Inside C's constructor."); }  
3 }
```

```
1 class CallingCons {  
2     public static void main(String[] args) {  
3         C c = new C();  
4     }  
5 }
```

An Example of Constructor Execution Order

```
1 class A {  
2     A() { IO.println("Inside A's constructor."); }  
3 }
```

```
1 class B extends A {  
2     B() { IO.println("Inside B's constructor."); }  
3 }
```

```
1 class C extends B {  
2     C() { IO.println("Inside C's constructor."); }  
3 }
```

```
1 class CallingCons {  
2     public static void main(String[] args) {  
3         C c = new C();  
4     }  
5 }
```

Console Output

```
Inside A's constructor.  
Inside B's constructor.  
Inside C's constructor.
```

Method Overriding

- In a **class hierarchy**, when a method in a **subclass** has the same name and type **signature** as a method in its **superclass**, then the method in the **subclass** is said to override the method in the **superclass**.
- When an **overridden** method is called through its **subclass**, it will always refer to the version of that method defined by the **subclass**.
- The version of the method defined by the **superclass** will be **hidden**.



An Example of Method Overriding

```
1 class A {  
2     int i, j;  
3     A(int a, int b) {  
4         i = a; j = b;  
5     }  
6     void show() {  
7         IO.println("i and j: "  
8             + i + " " + j);  
9     }  
10 }
```

```
1 class B extends A {  
2     int k;  
3     B(int a, int b, int c) {  
4         super(a, b); k = c;  
5     }  
6     void show() {  
7         IO.println("k: " + k);  
8     }  
9 }
```

```
1 class Override {  
2     public static void main(String[] args) {  
3         B subOb = new B(1, 2, 3);  
4         // this calls show() in B  
5         subOb.show();  
6     }  
7 }
```

An Example of Method Overriding

```
1 class A {
2     int i, j;
3     A(int a, int b) {
4         i = a; j = b;
5     }
6     void show() {
7         IO.println("i and j: "
8             + i + " " + j);
9     }
10 }
```

```
1 class B extends A {
2     int k;
3     B(int a, int b, int c) {
4         super(a, b); k = c;
5     }
6     void show() {
7         IO.println("k: " + k);
8     }
9 }
```

```
1 class Override {
2     public static void main(String[] args) {
3         B subOb = new B(1, 2, 3);
4         // this calls show() in B
5         subOb.show();
6     }
7 }
```

Console Output

k: 3

Access the Superclass Version of an Overridden Method

```
1 class B extends A {  
2     int k;  
3     B(int a, int b, int c) { super(a, b); k = c; }  
4     void show() {  
5         super.show(); // this calls A's show()  
6         IO.println("k: " + k);  
7     }  
8 }
```

Console Output

```
i and j: 1 2  
k: 3
```

Access the Superclass Version of an Overridden Method

```
1 class B extends A {  
2     int k;  
3     B(int a, int b, int c) { super(a, b); k = c; }  
4     void show() {  
5         super.show(); // this calls A's show()  
6         IO.println("k: " + k);  
7     }  
8 }
```

Console Output

```
i and j: 1 2  
k: 3
```

Method Overriding vs Overloading

```
1 class A {  
2     int i, j;  
3     A(int a, int b) { i = a; j = b; }  
4     void show() { IO.println("i and j: " + i + " " + j); }  
5 }
```

```
1 class B extends A {  
2     int k;  
3     B(int a, int b, int c) { super(a, b); k = c; }  
4     void show(String msg) { IO.println(msg + k); }  
5 }
```

```
1 class Override {  
2     public static void main(String[] args) {  
3         B subOb = new B(1, 2, 3);  
4         subOb.show("This is k: "); // this calls show() in B  
5         subOb.show(); // this calls show() in A  
6     }  
7 }
```

Dynamic Method Dispatch

- Method **overriding** forms the basis for one of Java's most powerful concepts: **dynamic method dispatch**.
- **Dynamic method dispatch** is the mechanism by which a call to an **overridden** method is resolved at **run time**, rather than compile time.
- **Dynamic method dispatch** is important because this is how Java implements **run-time polymorphism**.



Dynamic Method Dispatch

- When an **overridden** method is called through a **superclass reference**, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.
- When different types of objects are referred to, different versions of an **overridden** method will be called.



Dynamic Method Dispatch

- In other words, it is the type of the object being referred to (not the type of the reference variable) that determines which version of an **overridden** method will be executed.
- Therefore, if a superclass contains a method that is **overridden** by a **subclass**, then when different types of objects are referred to through a **superclass reference** variable, different versions of the method are executed.

An Example of Dynamic Method Dispatch

```
1 class A {  
2     void callme() {  
3         IO.println("Inside A's callme method");  
4     }  
5 }
```

```
1 class B extends A {  
2     void callme() {  
3         IO.println("Inside B's callme method");  
4     }  
5 }
```

```
1 class C extends A {  
2     void callme() {  
3         IO.println("Inside C's callme method");  
4     }  
5 }
```

An Example of Dynamic Method Dispatch

```
1 class Dispatch {  
2     public static void main(String[] args) {  
3         A a = new A(); // object of type A  
4         B b = new B(); // object of type B  
5         C c = new C(); // object of type C  
6         A r = a; // r refers to an A object  
7         r.callme(); // calls A's version of callme  
8         r = b; // r refers to a B object  
9         r.callme(); // calls B's version of callme  
10        r = c; // r refers to a C object  
11        r.callme(); // calls C's version of callme  
12    }  
13 }
```

Console Output

Inside A's callme method
Inside B's callme method
Inside C's callme method

An Example of Dynamic Method Dispatch

```
1 class Dispatch {  
2     public static void main(String[] args) {  
3         A a = new A(); // object of type A  
4         B b = new B(); // object of type B  
5         C c = new C(); // object of type C  
6         A r = a; // r refers to an A object  
7         r.callme(); // calls A's version of callme  
8         r = b; // r refers to a B object  
9         r.callme(); // calls B's version of callme  
10        r = c; // r refers to a C object  
11        r.callme(); // calls C's version of callme  
12    }  
13 }
```

Console Output

Inside A's callme method
Inside B's callme method
Inside C's callme method

An Example of Dynamic Method Dispatch

```
1 class Dispatch {  
2     public static void main(String[] args) {  
3         A a = new A(); // object of type A  
4         B b = new B(); // object of type B  
5         C c = new C(); // object of type C  
6         A r = a; // r refers to an A object  
7         r.callme(); // calls A's version of callme  
8         r = b; // r refers to a B object  
9         r.callme(); // calls B's version of callme  
10        r = c; // r refers to a C object  
11        r.callme(); // calls C's version of callme  
12    }  
13 }
```

Console Output

Inside A's callme method
Inside B's callme method
Inside C's callme method

An Example of Dynamic Method Dispatch

```
1 class Dispatch {  
2     public static void main(String[] args) {  
3         A a = new A(); // object of type A  
4         B b = new B(); // object of type B  
5         C c = new C(); // object of type C  
6         A r = a; // r refers to an A object  
7         r.callme(); // calls A's version of callme  
8         r = b; // r refers to a B object  
9         r.callme(); // calls B's version of callme  
10        r = c; // r refers to a C object  
11        r.callme(); // calls C's version of callme  
12    }  
13 }
```

Console Output

Inside A's callme method
Inside B's callme method
Inside C's callme method

Why Overridden Methods?

- As stated earlier, **overridden** methods allow Java to support **run-time polymorphism**.
- **Polymorphism** is essential to object-oriented programming for *one reason*: it allows a general class to specify methods that will be common to all of its derivatives, while allowing **subclasses** to define the specific implementation of some or all of those methods.



Why Overridden Methods?

- **Overridden** methods are another way that Java implements the “*one interface, multiple methods*” aspect of **polymorphism**.
- Dynamic, **run-time polymorphism** is one of the most powerful mechanisms that object-oriented design brings to bear on code **reuse** and **robustness**.
- The ability of **existing code** libraries to call methods on instances of new classes *without recompiling* while maintaining a clean abstract interface is a profoundly powerful tool.



Applying Method Overriding

```
1 class Figure {
2     double dim1; double dim2;
3     Figure(double a, double b) {
4         dim1 = a; dim2 = b;
5     }
6     double area() {
7         IO.println("Area for " +
8             "Figure is undefined.");
9         return 0;
10    }
11 }
```

```
1 class Rectangle extends Figure {
2     Rectangle(double a, double b) {
3         super(a, b);
4     }
5     double area() {
6         IO.println("Inside Area" +
7             " for Rectangle.");
8         return dim1 * dim2;
9     }
10 }
```

```
1 class Triangle extends Figure {
2     Triangle(double a, double b) {
3         super(a, b); }
4     double area() {
5         IO.println("Inside Area for Triangle.");
6         return dim1 * dim2 / 2;
7     }
8 }
```

Applying Method Overriding

```
1 class FindAreas {  
2     public static void main(String[] args) {  
3         Figure f = new Figure(10, 10);  
4         Rectangle r = new Rectangle(9, 5);  
5         Triangle t = new Triangle(10, 8);  
6         Figure figref;  
7         figref = r;  
8         IO.println("Area is " + figref.area());  
9         figref = t;  
10        IO.println("Area is " + figref.area());  
11        figref = f;  
12        IO.println("Area is " + figref.area());  
13    }  
14 }
```

Console Output

Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
Area for Figure is undefined.
Area is 0.0

Applying Method Overriding

```
1 class FindAreas {  
2     public static void main(String[] args) {  
3         Figure f = new Figure(10, 10);  
4         Rectangle r = new Rectangle(9, 5);  
5         Triangle t = new Triangle(10, 8);  
6         Figure figref;  
7         figref = r;  
8         IO.println("Area is " + figref.area());  
9         figref = t;  
10        IO.println("Area is " + figref.area());  
11        figref = f;  
12        IO.println("Area is " + figref.area());  
13    }  
14 }
```

Console Output

Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
Area for Figure is undefined.
Area is 0.0

Applying Method Overriding

```
1 class FindAreas {  
2     public static void main(String[] args) {  
3         Figure f = new Figure(10, 10);  
4         Rectangle r = new Rectangle(9, 5);  
5         Triangle t = new Triangle(10, 8);  
6         Figure figref;  
7         figref = r;  
8         IO.println("Area is " + figref.area());  
9         figref = t;  
10        IO.println("Area is " + figref.area());  
11        figref = f;  
12        IO.println("Area is " + figref.area());  
13    }  
14 }
```

Console Output

Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
Area for Figure is undefined.
Area is 0.0

Applying Method Overriding

```
1 class FindAreas {  
2     public static void main(String[] args) {  
3         Figure f = new Figure(10, 10);  
4         Rectangle r = new Rectangle(9, 5);  
5         Triangle t = new Triangle(10, 8);  
6         Figure figref;  
7         figref = r;  
8         IO.println("Area is " + figref.area());  
9         figref = t;  
10        IO.println("Area is " + figref.area());  
11        figref = f;  
12        IO.println("Area is " + figref.area());  
13    }  
14 }
```

Console Output

Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
Area for Figure is undefined.
Area is 0.0

Using Abstract Classes

- You can force **subclasses** to **override** specific methods by declaring them as **abstract**.
- These methods are sometimes referred to as *subclass responsibility* because they have *no implementation* specified in the **superclass**.
- Thus, a **subclass** must **override** them—it cannot simply use the version defined in the **superclass**.



Using Abstract Classes

- To declare an **abstract** method, use this general form:

```
abstract type name(parameter-list);
```

- As you can see, *no method body* is present.
- Any class that contains one or more **abstract** methods must also be declared **abstract**.
- To declare a class **abstract**, use this general form:

```
abstract class ClassName { ... }
```

Using Abstract Classes

- There can be no objects of an **abstract** class. That is, an **abstract** class cannot be directly instantiated with the `new` operator.
- Also, you cannot declare **abstract** constructors, or **abstract** static methods.
- Any **subclass** of an **abstract** class must either implement all of the **abstract** methods in the **superclass**, or be declared **abstract** itself.



Using Abstract Classes

- Although **abstract** classes cannot be *instantiated*, they can be used to declare object *references*.
- This is because Java implements **run-time polymorphism** using **superclass references**.
- As a result, a reference of an **abstract** class type can point to an object of any of its **subclasses**.

An Example of an Abstract Class

```
1 abstract class A {  
2     abstract void callme();  
3     void callmetoo() {  
4         IO.println("This is a concrete method.");  
5     }  
6 }
```

```
1 class B extends A {  
2     void callme() {  
3         IO.println("B's implementation of callme.");  
4     }  
5 }
```

```
1 class AbstractDemo {  
2     public static void main(String[] args) {  
3         B b = new B();  
4         b.callme();  
5         b.callmetoo();  
6     }  
7 }
```

Applying Abstract Classes

```
1 abstract class Figure {  
2     double dim1; double dim2;  
3     Figure(double a, double b) { dim1 = a; dim2 = b; }  
4     abstract double area();  
5 }
```

```
1 class Rectangle extends Figure {  
2     Rectangle(double a, double b) { super(a, b); }  
3     double area() {  
4         IO.println("Inside Area for Rectangle.");  
5         return dim1 * dim2;  
6     }  
7 }
```

```
1 class Triangle extends Figure {  
2     Triangle(double a, double b) { super(a, b); }  
3     double area() {  
4         IO.println("Inside Area for Triangle.");  
5         return dim1 * dim2 / 2;  
6     }  
7 }
```

Applying Abstract Classes

```
1 class AbstractAreas {
2     public static void main(String[] args) {
3         // Figure f = new Figure(10, 10); // illegal now
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref; // this is OK, no object is created
7         figref = r;
8         IO.println("Area is " + figref.area());
9         figref = t;
10        IO.println("Area is " + figref.area());
11    }
12 }
```

Applying Abstract Classes

```
1 class AbstractAreas {
2     public static void main(String[] args) {
3         // Figure f = new Figure(10, 10); // illegal now
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref; // this is OK, no object is created
7         figref = r;
8         IO.println("Area is " + figref.area());
9         figref = t;
10        IO.println("Area is " + figref.area());
11    }
12 }
```

Applying Abstract Classes

```
1 class AbstractAreas {
2     public static void main(String[] args) {
3         // Figure f = new Figure(10, 10); // illegal now
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref; // this is OK, no object is created
7         figref = r;
8         IO.println("Area is " + figref.area());
9         figref = t;
10        IO.println("Area is " + figref.area());
11    }
12 }
```

Applying Abstract Classes

```
1 class AbstractAreas {
2     public static void main(String[] args) {
3         // Figure f = new Figure(10, 10); // illegal now
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref; // this is OK, no object is created
7         figref = r;
8         IO.println("Area is " + figref.area());
9         figref = t;
10        IO.println("Area is " + figref.area());
11    }
12 }
```

Applying Abstract Classes

```
1 class AbstractAreas {
2     public static void main(String[] args) {
3         // Figure f = new Figure(10, 10); // illegal now
4         Rectangle r = new Rectangle(9, 5);
5         Triangle t = new Triangle(10, 8);
6         Figure figref; // this is OK, no object is created
7         figref = r;
8         IO.println("Area is " + figref.area());
9         figref = t;
10        IO.println("Area is " + figref.area());
11    }
12 }
```

Using *final* with Inheritance

- The keyword *final* has three uses.
- **First**, it can be used to create the equivalent of a named **constant**. This use was described in the preceding chapter.
- The other **two** uses of *final* apply to **inheritance**.

Using **final** to Prevent Overriding

- While method **overriding** is one of Java's most powerful features, there will be times when you will want to prevent it from occurring.
- To disallow a method from being **overridden**, specify **final** as a modifier at the start of its declaration.
- Methods declared as **final** cannot be **overridden**.

Using ***final*** to Prevent Overriding

- Methods declared as ***final*** can sometimes provide a performance enhancement.
- The **compiler** is free to inline calls to them because it “*knows*” they will **not** be **overridden** by a subclass.
- Normally, Java resolves calls to methods **dynamically**, at run time. This is called *late binding*.
- However, since ***final*** methods cannot be **overridden**, a call to one can be resolved at **compile time**. This is called *early binding*.

An Example of Using `final` to Prevent Overriding

```
1 class A {  
2     final void meth() {  
3         IO.println("This is a final method.");  
4     }  
5 }
```

```
1 class B extends A {  
2     void meth() { // ERROR! Can't override.  
3         IO.println("Illegal!");  
4     }  
5 }
```

Using ***final*** to Prevent Inheritance

- Sometimes you will want to prevent a **class** from being *inherited*.
- To do this, precede the **class** declaration with **final**.
- Declaring a **class** as **final** implicitly declares all of its methods as **final**, too.
- As you might expect, it is **illegal** to declare a **class** as both **abstract** and **final** since an **abstract class** is incomplete by itself and relies upon its subclasses to provide complete implementations.

An Example of Using `final` to Prevent Inheritance

```
1 final class A {  
2     //...  
3 }  
4 // The following class is illegal.  
5 class B extends A { // ERROR! Can't subclass A  
6     //...  
7 }
```

Local Variable Type Inference and Inheritance

- A **superclass** reference can refer to a **derived class** object, and this feature is part of Java's support for **polymorphism**.
- However, it is critical to remember that, when using **local variable type inference**, the inferred type of a variable is based on the declared type of its initializer.



Local Variable Type Inference and Inheritance

- Therefore, if the initializer is of the **superclass** type, that will be the inferred type of the variable.
- It does not matter if the actual object being referred to by the initializer is an instance of a **derived class**.

An Example of Local Variable Type Inference and Inheritance

```
1  class MyClass {  
2      // ...  
3  }  
4  class FirstDerivedClass extends MyClass {  
5      int x;  
6      // ...  
7  }  
8  class SecondDerivedClass extends FirstDerivedClass {  
9      int y;  
10     // ...  
11 }
```

An Example of Local Variable Type Inference and Inheritance

```
1  class TypeInferenceAndInheritance {
2      static MyClass getObj(int which) {
3          switch(which) {
4              case 0: return new MyClass();
5              case 1: return new FirstDerivedClass();
6              default: return new SecondDerivedClass();
7          }
8      }
9      public static void main(String[] args) {
10         var mc = getObj(0);
11         var mc2 = getObj(1);
12         var mc3 = getObj(2);
13     }
14 }
```

The Object Class

- There is one **special** class, **Object**, defined by Java.
- All other classes are **subclasses** of **Object**.
- That is, **Object** is a **superclass** of all other classes.
- This means that a reference variable of type **Object** can refer to an **object** of any other class.
- Also, since **arrays** are implemented as **classes**, a variable of type **Object** can also refer to any array.

Methods of the Object Class

Method	Purpose
<code>Object clone()</code>	Creates a new object that is the same as the object being cloned.
<code>boolean equals(Object object)</code>	Determines whether one object is equal to another.
<code>void finalize()</code>	Called before an unused object is recycled. (Deprecated by JDK 9.)
<code>Class<?> getClass()</code>	Obtains the class of an object at run time.
<code>int hashCode()</code>	Returns the hash code associated with the invoking object.

Methods of the Object Class

Method	Purpose
<code>void notify()</code>	Resumes execution of a thread waiting on the invoking object.
<code>void notifyAll()</code>	Resumes execution of all threads waiting on the invoking object.
<code>String toString()</code>	Returns a string that describes the object.
<code>void wait()</code>	Waits on another thread of execution.
<code>void wait(long milliseconds)</code>	
<code>void wait(long milliseconds, int nanoseconds)</code>	